



CONSIGNMENT BOUTIQUE

Classic Consigns by KYS

Point of Sale & Store Management · ConsignPro platform

User & Admin Guides

USER GUIDE

1. Signing in
2. Point of Sale (POS)
 - Ringing up a sale
 - Notes
3. Inventory
 - Views & sorting
 - The scan box
 - Adding an item
 - Editing an item
 - Bulk actions
 - Labels
 - Aging alerts
4. Consignors
 - Consignor detail page
5. Customers
6. Payouts
7. Contracts
8. Store Insights & Reports
9. Online shop
10. Consignor & Dealer portals
11. Importing existing data
- Quick tips

ADMIN & OPERATIONS GUIDE

1. Architecture at a glance
2. Hosting on Railway
 - Deploying changes
3. Environment variables
4. First-run setup (new deployment)
5. Accounts & roles
 - Resetting a password
6. Store configuration (Settings)
7. Integrations
 - Clover (card payments)
 - Resend (email)
 - Anthropic / Claude (AI item entry)
 - Shopify (webhooks, inbound)
 - Outbound webhooks / API keys
8. Importing data
9. Data export & reporting
10. Backups & data safety
11. Direct database access
12. Local development
13. Troubleshooting
14. Security checklist

Updated **October 2026** · Staff User Guide and Owner/Admin Guide

 7214 Harford Rd, Baltimore, MD 21234 ·  443.841.9730 ·  kimberly@classicconsigns.com

User Guide

For store staff

ConsignPro — User Guide

A guide for store staff (owners, managers, and employees) using the ConsignPro consignment platform day to day.

Your store address: your platform lives at your own web address (e.g. `https://your-store.up.railway.app`). Replace "your site" below with it.

1. Signing in

1. Go to **your site** → you'll land on the **Sign in** page.
2. Enter your **email** and **password** → **Sign In**.
3. You'll arrive at the **Point of Sale** screen.

Forgot your password? Ask your store owner/admin — they can reset it (see the Admin Guide → "Resetting a password").

Dark / light mode: at the bottom of the left sidebar, use **Dark mode / Light mode** to switch themes. Your choice is remembered on that device.

2. Point of Sale (POS)

The POS is where you ring up sales.

Ring up a sale

1. **Find items** — type a name or SKU in the search box, or **scan a barcode**. - **Scanning:** just scan the item's label — it's **added straight to the cart** and you'll see a green "Added: ..." confirmation. Scan the next item, and so on. - **Typing a name:** results appear below; click one to add it to the cart.
2. **Review the cart** on the right — remove a line with the trash icon.

3. **Discount (optional):** type a dollar amount in the Discount box.
4. **Loyalty points (optional):** - In the **Loyalty Points** box, type the customer's **email** → **Find**. - Their available points show. Enter how many points to **redeem** (100 points = \$1). The discount is applied automatically.
5. **Choose payment: Card** (Clover) or **Cash**.
6. Click **Charge \$X.XX**.
7. A **receipt** pops up — **Print Receipt** or Close. Points are awarded to the customer automatically (1 point per \$1 spent) when a customer is attached.

Notes

- Only **Active** items can be sold. Sold/returned items won't scan into a sale.
 - Card payments create an order in your Clover account; the actual card charge happens on your Clover device/terminal.
-

3. Inventory

Manage every item in the store. Open **Inventory** from the sidebar.

Views & sorting

- Top-right, switch between **List**, **Cards**, and **Compact** views.
- In **List** view, click any column header (Item, Brand, Consignor, Price, Added) to sort; click again to reverse.
- **Status tabs** (Active / Sold / Returned / Expired) filter what you see.
- **Search** by title, brand, SKU, or consignor name.

The scan box

At the top there's a **Scan** box. Scan a barcode or SKU (or type it and press Enter) and that item **opens for editing** immediately — handy for quick price checks or edits at the counter.

Adding an item

1. **Inventory** → **Add Item**.
2. Fill in the details: title, brand, size, condition, price, consignor, category.
3. **AI Item Entry (optional):** drop a photo and the assistant fills in the title, brand, category, condition, size, color, description, and a suggested price for you to review.
4. **Photos:** drag-and-drop photos; the first is the main image.
5. **Custom SKU (optional):** type your own SKU (e.g. `CC-DRESS-001`) or leave it blank to auto-generate.
6. **Barcode:** scan a barcode into this field (or leave blank to use the SKU).
7. **List on public online shop:** toggle on to show the item at your `/shop` .

8. **Save** → you'll get the SKU and a **Print Label** button.

Editing an item

Click any item (row or card) to open the **Edit** window. You can change the title, SKU, barcode, brand, size, condition, price, status, and online listing, **Print Label**, or **Delete** (deletion is blocked for items with sales).

Bulk actions

Select items with the checkboxes (or "select all"), then use the blue bar: - **Print labels** — prints a scannable label for every selected item. - **Markdown %** — reduce the price of all selected items by a percentage. - **List online / Unlist** — show/hide on the public shop in bulk. - **Mark Returned** — change status in bulk. - **Delete** — remove selected items (sold items can't be deleted).

Labels

Printed labels are **2" × 1"** and carry a **scannable Code128 barcode** (the item's barcode, or its SKU), the code, and the price. Print from the Add-Item screen, the Edit window, or the bulk **Print labels** action.

Aging alerts

On the Active tab, items expiring within 14 days are flagged at the top so you can mark them down or return them.

4. Consignors

The people who bring you items to sell. Open **Consignors** from the sidebar.

- **Views / sorting / search / multi-select** work the same as Inventory.
- **Add** a consignor with the **Add** button (name, contact, split %).
- **Click a consignor** to edit their name, contact, **split %**, and **portal access**, or open **Details** for their full items / earnings / payout history.
- **Bulk actions**: set split %, enable/disable portal access, delete.
- **Balance** is what you currently owe each consignor.
- When adding or editing a consignor you can set their **preferred payout method** (Check/Cash/Zelle/Cash App/ACH) with the matching details, and an **"If items don't sell"** preference: **Pick-up**, **Donate**, or **Continue consigning (30 days)**.

Consignor detail page

Shows KPI cards (balance, earned, active, sold), and tabs for **Items**, **Ledger** (every credit/debit), and **Payouts**. You can **issue a payout** here (Check / Cash / ACH).

5. Customers

Your loyalty members. Open **Customers** from the sidebar.

- **Views / sorting / search / multi-select** as above.
 - **Add** a customer (name, email, phone).
 - **Click a customer** to edit name, email, phone, and **loyalty points**, or delete them.
 - **Bulk actions:** add points to many at once, or delete.
 - Customers earn **1 point per \$1** spent (when attached to a sale) and redeem **100 points = \$1** at the POS.
-

6. Payouts

Pay your consignors. Open **Payouts** from the sidebar.

- **Pending Balances** tab lists consignors who are owed money.
 - Select who to pay, choose the **method** (Check, Cash, **Zelle**, **Cash App**, or ACH), and process — checks are numbered sequentially.
 - For **Zelle / Cash App / Check**, you'll be asked for the destination (their Zelle phone/email, Cash App \$Cashtag, or mailing address). It's saved on the consignor, so next time it's pre-filled. Batch payouts use each consignor's saved details automatically.
 - **Payout History** tab shows everything you've paid.
 - Consignors with portal access can also **request a payout** themselves; those requests appear as pending payouts for you to process.
-

7. Contracts

Consignment agreements with e-signature. Open **Contracts** from the sidebar.

1. **New Contract** → pick a consignor → optionally **send the signing link by email**.
2. The consignor opens the link, reads the agreement, **signs on screen**, and submits. Their signature, IP, and timestamp are recorded.
3. The contract's status moves **Draft** → **Sent** → **Signed**.
4. Click the eye icon to view a contract (and its signature).

The agreement text is your store's standard template; the owner can edit it in **Settings** → **Contract Template**.

8. Store Insights & Reports

- **Store Insights** — charts for 30-day revenue, top categories, and inventory status.
 - **Reports** — key numbers plus **CSV export** for Sales, Inventory, Consignors, and Ledger (with optional date range). Great for bookkeeping.
-

9. Online shop

Your public storefront is at **your site** `/shop` — no login needed; this is what customers see. Any item marked "**List on public online shop**" appears there automatically. Share that link on social media or your website.

10. Consignor & Dealer portals

- **Consignor portal** (`/portal`) — consignors with portal access sign in to see their items, earnings, and balance, and to **request a payout**.
 - **Dealer entry** (`/dealer`) — lets approved dealers submit multiple items at once for your review.
-

11. Importing existing data

Owners/managers can bulk-load data at **your site** `/import` by uploading a prepared file: - A **consignors + inventory** file, or - A **customers** file.

The page shows a preview (counts) before you confirm. Records are matched by email so nothing is duplicated. (Preparing these files from spreadsheets is done with help from your admin.)

Quick tips

- **Scanning works anywhere there's a field** — the scanner just "types" the code and presses Enter.
- **Switch views** (List / Cards / Compact) to suit the task — Compact is great for scanning long lists; Cards are nice for browsing with photos.
- **Multi-select + bulk actions** save a lot of time (markdowns, labels, points).
- Use **Dark mode** for low-light counters.

Questions about setup, accounts, payments, or integrations? See the Admin Guide.

Admin & Operations Guide

For the owner / administrator

ConsignPro — Admin & Operations Guide

For the store owner / administrator. Covers hosting, configuration, accounts, security, integrations, data import/export, backups, and troubleshooting.

1. Architecture at a glance

Layer	Technology
App	Next.js 16 (App Router, TypeScript), React 19
Database	PostgreSQL via Prisma 7 (pg driver adapter)
Auth (staff)	NextAuth v5 (JWT sessions, credentials)
Auth (consignor/dealer portal)	separate JWT (<code>jose</code>)
Hosting	Railway (app + Postgres + a Volume for photos)
Photo storage	Railway Volume (served via <code>/api/photos</code>)
Payments	Clover REST API
Email	Resend
AI item entry	Anthropic Claude (<code>claude-haiku-4-5</code>) vision
Barcodes	Code128 labels via JsBarcode

The code lives in the GitHub repo and **auto-deploys to Railway** on every push to the connected branch.

2. Hosting on Railway

The project (e.g. "patient-dream") contains two services: 1. **App service** — deployed from the GitHub repo. Has a **Volume** mounted (e.g. at `/data`) for item photos. 2. **Postgres** — the database.

Build/run config lives in `railway.json`: - Build: Nixpacks, `npm run build`. - Start: `npx prisma db push --accept-data-loss && npm run start` (syncs the database schema, then starts the server). - Healthcheck path: `/login`.

The app binds to Railway's `$PORT` automatically. The Prisma client is created lazily, so the **build** does not require a database connection, but the **running app** and the start command do (via `DATABASE_URL`).

Deploying changes

Push to the connected branch → Railway rebuilds and redeploys automatically. Watch the **Deployments** tab; a healthy deploy ends **Active / green**.

3. Environment variables

Set these on the **app service** → **Variables** tab. Required unless noted.

Variable	Purpose
<code>DATABASE_URL</code>	Postgres connection. Use the reference <code>{{Postgres.DATABASE_URL}}</code> . Must be set before the first deploy.
<code>NEXTAUTH_URL</code>	The app's public URL, e.g. <code>https://your-store.up.railway.app</code> .
<code>NEXTAUTH_SECRET</code>	Random secret for staff auth. Generate: <code>openssl rand -base64 32</code> (or <code>node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"</code>).
<code>PORTAL_JWT_SECRET</code>	Random secret for consignor/dealer portal tokens.
<code>ANTHROPIC_API_KEY</code>	<i>Optional.</i> Enables AI item entry. From console.anthropic.com.
<code>CLOVER_API_KEY</code> / <code>CLOVER_MERCHANT_ID</code>	<i>Optional.</i> Enables Clover card orders.
<code>CLOVER_API_BASE</code>	<i>Optional.</i> <code>https://sandbox.dev.clover.com</code> for sandbox testing; omit for production.
<code>RESEND_API_KEY</code> / <code>EMAIL_FROM</code>	<i>Optional.</i> Enables outbound email (contracts, payouts).
<code>SHOPIFY_WEBHOOK_SECRET</code>	<i>Optional.</i> Verifies Shopify webhooks.
<code>RESET_TOKEN</code>	<i>Optional/temporary.</i> Enables the <code>/reset</code> password-reset page while set. Remove when not in use.

A volume at a mount path (Railway sets `RAILWAY_VOLUME_MOUNT_PATH`) stores photos under `<mount>/uploads` . `UPLOAD_DIR` can override the location.

4. First-run setup (new deployment)

1. Set **all required** env vars (above), especially `DATABASE_URL` , `NEXTAUTH_URL` , `NEXTAUTH_SECRET` , `PORTAL_JWT_SECRET` .
2. Attach a **Volume** to the app service (e.g. mount `/data`).
3. Deploy; wait for **Active**.
4. Open **your site** `/api/setup` — it should return `{"needsSetup":true}` .
5. Open **your site** `/setup` → enter store name, your email, a password → **Create my account**. This creates the owner account and some starter categories, then **locks itself**.
6. Sign in at `/login` .

5. Accounts & roles

Roles: **OWNER**, **MANAGER**, **EMPLOYEE** (set on the `User` record).

- **Owner/Manager** can change store settings, manage API keys & webhooks, import data, and edit store/Clover settings.
- The first owner is created via `/setup`.
- There is no staff-invite UI yet. To add staff, create a `User` row in the database (see "Direct database access") with the store's `storeId`, a bcrypt-hashed password, and the desired role.

Resetting a password

1. Add a temporary env var `RESET_TOKEN` = any random string.
2. Wait for redeploy, then open **your site** `/reset`.
3. Enter the token, the account email, and a new password → **Update password**.
4. **Remove the `RESET_TOKEN` variable** afterward to disable the page.

6. Store configuration (Settings)

Settings (sidebar) covers: - **Store Information** — name, email, phone, **tax rate** (entered as %), currency. Used across the app (e.g. POS tax). - **Clover** — merchant ID + API key (stored in the database; env vars also work as a fallback). - **Online Shop** — your public `/shop` URL. - **Categories** — add/manage item categories. - **API Keys** — generate keys for external integrations (shown once). - **Webhooks** — register endpoints to receive events (sale.created, item.sold, contract.signed, etc.), HMAC-signed. - **Discount Rules** — percentage or fixed-amount rules, optional promo codes. - **Contract Template** — edit the consignment agreement. Placeholders: `{{storeName}}`, `{{consignorName}}`, `{{splitPercent}}`.

7. Integrations

Clover (card payments)

Set `CLOVER_API_KEY` + `CLOVER_MERCHANT_ID` (env or Settings). Use

`CLOVER_API_BASE=https://sandbox.dev.clover.com` for sandbox testing. **Note:** the app creates Clover **orders**; the actual card capture happens on a Clover **device/terminal** tied to the merchant — full in-app card capture is not yet implemented.

Resend (email)

Set `RESEND_API_KEY` and `EMAIL_FROM` (verified domain recommended). Powers contract links, payout notifications, and welcome emails.

Anthropic / Claude (AI item entry)

Set `ANTHROPIC_API_KEY`. The add-item photo analysis uses `claude-haiku-4-5`. Cost is well under a cent per item. If unset, AI entry simply returns a 503 and everything else works.

Shopify (webhooks, inbound)

Set `SHOPIFY_WEBHOOK_SECRET`; point Shopify order webhooks at `/api/webhooks/shopify`. Incoming orders mark matched items sold and credit the consignor.

Outbound webhooks / API keys

Configure in **Settings**. API requests authenticate with a `Bearer cp_live_...` key; only a SHA-256 hash is stored.

8. Importing data

Owners/managers upload prepared JSON at **your site** `/import` : - `{ consignors: [...] }` — creates consignors (matched by email) and their items (Active, priced from the sheet, default 50% split). - `{ customers: [...] }` — creates customers (matched by email).

Both are idempotent (safe to re-run; existing records are skipped). Preparing these files from spreadsheets is typically done by cleaning the sheet into the expected JSON shape; ask the maintainer to generate the import file from a new spreadsheet.

9. Data export & reporting

Reports → **Export** produces CSVs for Sales, Inventory, Consignors, and Ledger (optional date range) for bookkeeping and accounting.

10. Backups & data safety

- **Database:** enable Railway's Postgres **backups/snapshots**. The database holds all consignors, customers, items, sales, ledger, payouts, and contracts.

- **Photos:** the Railway **Volume** holds item photos — enable volume snapshots. Photos are **not** in git or the database.
- **Deletions are protected:** consignors/items with sales history can't be deleted.

The start command uses `prisma db push --accept-data-loss`. The current schema is additive, so this is a no-op in practice. Before making a **destructive** schema change against live data, take a database backup first, and consider switching to committed Prisma migrations (`prisma migrate`).

11. Direct database access

- Browse/edit data with Prisma Studio locally: `DATABASE_URL="..." npx prisma studio`.
- Railway also provides a **Console** and a database connection string for the Postgres service.

12. Local development

See `LOCAL_DEV.md` for the full local setup (clone, `.env`, `docker compose up` for a local Postgres, `npm run db:push`, `npm run db:seed`, `npm run dev`).

13. Troubleshooting

Symptom	Likely cause / fix
Build fails with <code>P2038 ... driver adapter</code>	A DB route evaluated at build time without <code>DATABASE_URL</code> . The client is lazy now; ensure you're on current code.
"Application failed to respond" after deploy	The start command failed — check Deploy Logs . Often the DB migration step or a bad start command.
Every page 500s	Missing <code>NEXTAUTH_SECRET</code> , or NextAuth can't trust the host — <code>trustHost</code> is enabled; make sure the secret is set.
Login says "invalid email or password" with a correct password	Check the account exists (<code>/api/setup</code> → <code>needsSetup:false</code>). If needed, reset via <code>/reset</code> .
Login succeeds then bounces to <code>/login</code>	Session cookie not seen on navigation — the app uses a hard navigation after login to prevent this; ensure current code.
<code>/api/setup</code> returns 500	Database has no tables / can't connect. The start command runs <code>prisma db push</code> ; verify <code>DATABASE_URL</code> .
AI item entry returns 503	<code>ANTHROPIC_API_KEY</code> not set (optional feature).
Photos don't upload/show	Volume not attached, or <code>/api/photos</code> blocked. Ensure a Volume is mounted.
Card payments don't charge	Expected — Clover orders are created, but card capture needs a Clover device.

Reading logs: Railway → app service → latest deployment → **View logs** → **Deploy Logs**. Use the filter box to find specific text.

14. Security checklist

- ☐ `NEXTAUTH_SECRET` and `PORTAL_JWT_SECRET` are strong, unique random values.
 - ☐ `RESET_TOKEN` is **removed** when not actively resetting a password.
 - ☐ Database and Volume backups are enabled.
 - ☐ The owner password was changed from any initial value.
 - ☐ Secrets are only in Railway Variables (never committed to git).
 - ☐ API keys are rotated if exposed.
-

See `USER_GUIDE.md` for day-to-day staff instructions.